Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of Electrical & Electronic Engineering

Rajshahi-6204, Bangladesh

Capstone Project Design I Report

Course Code: EEE 3202

Project Title:

Real-Time Edge-Based Facial Recognition System Using Deep Learning on Raspberry Pi 4

Submitted by:	Name: Labib Marwan Hoque Roll No.: 2101064 Department of EEE, RUET
Submitted to:	Supervisor's Name: Md. Mayenul Islam Designation: Assistant Professor Department of EEE, RUET

Date of Project Showcasing: 12.01.2026

Date of Report Submission: 26.04.2026

Academic Year: 2025–2026

Contents

Real-Time Edge-Based Facial Recognition System Using Deep Learning on Raspberry Pi 4

Capstone Project Design I (EEE 3202) — Final Report

Project Member:	Labib Marwan Hoque (Roll: 2101064)
Department:	Electrical & Electronic Engineering, RUET
Supervisor:	Md. Mayenul Islam, Assistant Professor, EEE, RUET
Course:	EEE 3202 — Capstone Project Design I

Abstract

This report presents the design, implementation, and evaluation of a real-time, offline facial recognition system running entirely on a Raspberry Pi 4 Model B (4 GB RAM). Existing cloud-based biometric systems raise serious concerns regarding data privacy, network latency, and operational cost. At the same time, conventional edge-based approaches (e.g., Dlib/HOG) fail to deliver usable frame rates on cost-effective single-board computers without expensive hardware accelerators. This project addresses the gap by deploying a quantized deep-learning pipeline — **YuNet** for face detection and **SFace** for recognition — via OpenCV's native DNN module. The system achieves a stable frame rate of 3–4 FPS with an inference latency below 300 ms per frame, is entirely offline, and supports instant "One-Shot" user registration through a dedicated `add_face.py` script. Experimental results confirm robust detection up to $\pm 30^{\circ}$ yaw and simultaneous multi-face tracking of both registered and unregistered individuals.

1 Introduction

1.1 Background and Motivation

Biometric security and smart automation are rapidly becoming indispensable components of modern infrastructure — from secure office access to automated attendance systems in educational institutions. Facial recognition is particularly appealing as a biometric modality because it is contactless, non-intrusive, and requires no active cooperation from the user.

Most deployed facial recognition systems today rely on **cloud computing**: video frames are streamed to a remote server for inference, and results are transmitted back to the local device. This architecture introduces three critical problems:

1. **Data Privacy:** Biometric data — the most personal form of user data — is transmitted to, and stored on, third-party servers. This creates significant legal and ethical risks, particularly under emerging data-protection legislation.

2. **Network Latency:** The round-trip time imposed by internet connectivity introduces unacceptable lag for real-time access-control applications.

3. **Operational Cost:** Continuous cloud API usage incurs ongoing financial costs that are prohibitive for small institutions and developing economies.

Edge-computing alternatives — running inference locally on the device — avoid all three problems but have historically been limited by hardware constraints. Classical algorithms such as **Haar Cascades** are computationally light but suffer from poor accuracy in non-frontal views or challenging lighting. Modern deep-learning approaches such as **Dlib/HOG** are accurate but computationally too expensive for commodity single-board computers (SBCs) like the Raspberry Pi, routinely yielding frame rates below 1 FPS.

1.2 Research Gap

Current literature explores lightweight models such as MobileNet and SqueezeNet, yet deployment on general-purpose SBCs typically requires specialized neural accelerators (e.g., Google Coral USB Accelerator or NVIDIA Jetson). There is a distinct lack of optimised *software-only* pipelines that can perform real-time deep-learning inference on a standard Raspberry Pi 4 using only its native ARM CPU.

1.3 Proposed Solution

This project addresses the gap by implementing a **quantized deep-learning pipeline** using OpenCV's DNN module. The pipeline combines:

- **YuNet** — a quantized, nanometre-scale face detector optimised for ARM edge devices.
- **SFace** — a Sigmoid-Constrained Hypersphere Loss recognition model producing compact 128-dimensional embedding vectors.

Together, these models enable a secure, offline, real-time biometric system that runs entirely on the edge, without any cloud dependency or external accelerator hardware.

2 Objectives

The primary objective of this project is to develop a standalone, offline facial recognition system on a Raspberry Pi 4. The specific aims are:

1. **Objective 1:** To implement an optimised deep-learning pipeline (YuNet + SFace) on the Raspberry Pi 4 using OpenCV's DNN module.
2. **Objective 2:** To achieve a usable real-time frame rate of 3–5 FPS on the native ARM CPU, without any external hardware accelerators.
3. **Objective 3:** To develop a robust "One-Shot" encoding system (`add_face.py`) that allows new users to be registered instantly — capturing 15 face variations, computing embeddings, and updating the database — without re-training the model.
4. **Objective 4:** To implement dynamic tracking of unregistered individuals ("Unknown" faces) alongside registered users for simultaneous security monitoring.
5. **Objective 5:** To demonstrate the system's relevance to national development goals (Digital Bangladesh / Smart Bangladesh 2041) as a low-cost, domestically producible security solution.

3 Theoretical Background

3.1 Face Detection: YuNet

YuNet is a lightweight convolutional neural network specifically designed for face detection on resource-constrained edge devices. Its key design principles are:

- **Quantized weights:** The model is quantized to INT8 precision, drastically reducing the number of floating-point operations per inference.
- **Small input tensor:** Operating on a resized 320×320 frame instead of the full 640×480 resolution reduces computation by approximately 75%.
- **Facial landmark detection:** In addition to the bounding box, YuNet outputs five facial landmarks (two eye corners, nose tip, two mouth corners), enabling geometric alignment before recognition.
- **ARM optimisation:** The ONNX model is compiled with ARM NEON intrinsics in OpenCV's DNN backend, exploiting SIMD parallelism on the Raspberry Pi's Cortex-A72 cores.

The detection output for a single face is a vector of the form:

$$\mathbf{d} = [x,\ y,\ w,\ h,\ x_{re},\ y_{re},\ x_{le},\ y_{le},\ x_n,\ y_n,\ x_{rm},\ y_{rm},\ x_{lm},\ y_{lm},\ \text{score}]$$

where (x, y, w, h) is the bounding box, the next ten values are the five landmark coordinates, and `score` is the detection confidence.

3.2 Face Recognition: SFace

SFace (Sigmoid-Constrained Hypersphere Loss) is a face recognition model that maps an aligned face crop to a 128-dimensional unit-norm embedding vector $\mathbf{f} \in \mathbb{R}^{128}$, $\|\mathbf{f}\|_2 = 1$. Recognition is performed by comparing the cosine similarity between a live embedding and stored reference embeddings:

$$S_{\cos}(\mathbf{f}_1, \mathbf{f}_2) = \frac{\mathbf{f}_1 \cdot \mathbf{f}_2}{\|\mathbf{f}_1\|_2 \cdot \|\mathbf{f}_2\|_2} \tag{1}$$

A match is declared if $S_{\cos} \geq \tau$, where the threshold $\tau = 0.363$ is the standard value recommended for the SFace ONNX model. This threshold was derived from the model's Equal Error Rate (EER) on the LFW benchmark dataset. A higher threshold increases security (fewer false acceptances) at the cost of more false rejections.

3.3 One-Shot Encoding

Unlike training-based approaches (which require hundreds of images per person and hours of GPU compute to update the model), this system uses one-shot encoding: a small set of reference images ($\sim$15 per person) is captured, embeddings are extracted and stored in a serialised `.pickle` database, and recognition proceeds by nearest-neighbour cosine-similarity lookup at runtime. No model retraining is required.

3.4 Program Outcomes Addressed

Table 1: Program Outcomes (PO) addressed by this project

PO	Category	How Addressed
PO(e)	Modern Tool Usage	Python 3.11, OpenCV 4.6.0 DNN module, ONNX model format, Picamera2 library, NumPy for vector mathematics.
PO(i)	Individual Work	End-to-end system lifecycle: research into model quantization, Linux (Bookworm OS) environment configuration, hardware integration, and software optimisation.
PO(j)	Communication	Technical poster, block diagrams, performance analysis charts (FPS, latency), and a visual UI with colour-coded bounding boxes and confidence scores.
PO(k)	Project Management	Strict budget (BDT 16,600) using off-the-shelf components; academic timeline management across Design $\rightarrow$ Implement $\rightarrow$ Test lifecycle.

4 Apparatus and Cost Estimation

4.1 Hardware Components

Table 2: Required Hardware Components with Cost

No.	Component	Qty.	Unit Price (BDT)	Total Cost (BDT)
1	Raspberry Pi 4 Model B (4 GB RAM)	1	12,850	12,850
2	Raspberry Pi Camera Module V1.3 (5 MP, OV5647)	1	880	880
3	Micro SD Card (32 GB, Class 10)	1	600	600
4	Power Bank (5V / 3A, USB-C)	1	2,000	2,000
5	USB-A to USB-C Cable	1	150	150
6	Aluminium Heat Sink Set with Thermal Adhesive	1	120	120
			Total Expense	**16,600**

4.2 Software Stack

Table 3: Software Components

No.	Component	Version	Role
1	Raspberry Pi OS Bookworm	Debian 12 (64-bit)	Operating System
2	Python	3.11	Core programming language
3	OpenCV (DNN Module)	4.6.0	Deep learning inference engine
4	Picamera2	Latest (apt)	Low-latency CSI camera driver
5	NumPy	1.24+	Vector mathematics and array operations
6	YuNet ONNX Model	2022mar	Face detector
7	SFace ONNX Model	2021dec	Face recognizer

5 Methodology

5.1 Hardware Setup

The Raspberry Pi 4 Model B (4 GB) was connected to the 5 MP OV5647 Camera Module via the CSI (Camera Serial Interface) ribbon cable. The blue side of the ribbon faces the USB ports. Active cooling using an aluminium heat sink set was applied to prevent thermal throttling during sustained AI inference, which can otherwise cause the Cortex-A72 to reduce its clock speed from 1.8 GHz to 1.5 GHz.
The operating system was Raspberry Pi OS Bookworm (Debian 12, 64-bit), which uses `libcamera` as the default camera stack. The Legacy Camera interface was **disabled** in `raspi-config` to avoid conflicts with Picamera2.

5.2 Dependency Installation

All required packages were installed from the system repositories to avoid lengthy compilation (Bookworm's system Python is managed by `apt`):

Listing 1: Code 1: System and Python dependency installation on Bookworm

```
sudo apt update
sudo apt install -y python3-opencv python3-picamera2 python3-numpy
```

The YuNet and SFace ONNX model files were downloaded from the OpenCV Model Zoo:

Listing 2: Code 2: Model file download

```
cd ~/face_project

# Face Detector (YuNet) - compatible with OpenCV 4.6.0
wget https://github.com/opencv/opencv_zoo/raw/main/models/\
face_detection_yunet/face_detection_yunet_2022mar.onnx

# Face Recognizer (SFace)
wget https://github.com/opencv/opencv_zoo/raw/main/models/\
face_recognition_sface/face_recognition_sface_2021dec.onnx
```

Note: The 2023mar version of YuNet requires OpenCV $\geq$ 4.7.0 and is **incompatible** with the 4.6.0 build available via `apt` on Bookworm. The 2022mar model is fully compatible.

5.3 System Processing Pipeline

The system follows a five-stage linear pipeline on every captured frame. The pipeline is summarised in Fig. 1.

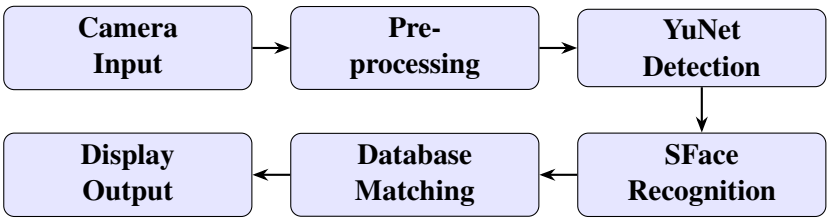

Fig. 1: Five-stage real-time processing pipeline.

1. **Image Acquisition:** The Pi Camera V1.3 streams video at 640×480 pixels in BGR888 colour format via Picamera2. BGR888 provides colour accuracy for the AI models without a colour-space conversion step.

2. **Pre-processing:** Each frame is horizontally mirrored (`cv2.flip(..., 1)`) for natural user experience (like looking into a mirror). The detector's input size is then set to 320×320 to match its expected input tensor, reducing inference load.

3. **Face Detection (YuNet):** The pre-processed frame is passed to the YuNet detector, which returns bounding boxes and five facial landmark coordinates for each detected face above the confidence threshold of 0.8.

4. **Face Recognition (SFace):** Each detected face is geometrically aligned using the five landmarks (`recognizer.alignCrop()`) to normalise for pose variations, then passed through SFace to produce a 128-dimensional embedding vector.

5. **Matching and Display:** The live embedding is compared against all stored embeddings in `encodings.pickle` using cosine similarity. If the best match score exceeds $\tau = 0.363$, the face is labelled with the corresponding name and a green bounding box; otherwise it is marked "Unknown" with a red bounding box, and the confidence percentage is displayed.

5.4 Main Recognition Script

Listing 3: Code 3: `main_deeplearning.py` — Real-time recognition system

```
import cv2
import numpy as np
import os
import time
import pickle
from picamera2 import Picamera2

# --- CONFIGURATION ---
ENCODINGS_FILE      = 'encodings.pickle'
DETECTOR_MODEL      = 'face_detection_yunet_2022mar.onnx'
RECOGNIZER_MODEL    = 'face_recognition_sface_2021dec.onnx'
```

```
CONFIDENCE_THRESHOLD = 0.8     # YuNet detection confidence
MATCH_THRESHOLD      = 0.363   # SFace cosine similarity threshold

# Load AI Models
detector  = cv2.FaceDetectorYN.create(
    DETECTOR_MODEL, "", (320, 320), CONFIDENCE_THRESHOLD, 0.3, 5000)
recognizer = cv2.FaceRecognizerSF.create(RECOGNIZER_MODEL, "")

# Load pre-computed embeddings database
with open(ENCODINGS_FILE, "rb") as f:
    data = pickle.loads(f.read())
known_feats = data["feats"]
known_names = data["names"]
print(f"Loaded {len(known_names)} known face embeddings.")

# Setup Camera
picam2 = Picamera2()
config = picam2.create_preview_configuration(
    main={"size": (640, 480), "format": "BGR888"})
picam2.configure(config)
picam2.start()
time.sleep(2)  # Warm-up

print("Running. Press 'q' to quit.")

while True:
    frame = picam2.capture_array()
    frame = cv2.flip(frame, 1)       # Mirror for UX

    height, width, _ = frame.shape
    detector.setInputSize((width, height))

    _, faces = detector.detect(frame)    # YuNet inference

    if faces is not None:
        for face in faces:
            box          = face[0:4].astype(int)
            aligned_face = recognizer.alignCrop(frame, face)
            feat         = recognizer.feature(aligned_face)

            name      = "Unknown"
            max_score = 0.0

            for i, known_feat in enumerate(known_feats):
                score = recognizer.match(
                    known_feat, feat, cv2.FaceRecognizerSF_FR_COSINE)
                if score > MATCH_THRESHOLD and score > max_score:
                    max_score = score
                    name = known_names[i]

            color = (0, 255, 0) if name != "Unknown" else (0, 0, 255)
            x, y, w, h = box
            cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)
            label = f"{name} ({int(max_score*100)}%)"
            cv2.rectangle(frame, (x, y+h-28), (x+w, y+h), color, cv2.FILLED)
            cv2.putText(frame, label, (x+4, y+h-6),
                        cv2.FONT_HERSHEY_DUPLEX, 0.6, (255,255,255), 1)

    cv2.imshow('Pi 4 Face Recognition System', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

picam2.stop()
cv2.destroyAllWindows()
```

5.5 One-Shot User Registration Script

A dedicated script (`add_face.py`) handles user enrolment without requiring model retraining. When invoked, it: (i) prompts for the user's name, (ii) opens the camera and captures 15 face images with a 2-second countdown between each, (iii) immediately computes SFace embeddings for all captured images, and (iv) appends the new embeddings to `encodings.pickle`.

Listing 4: Code 4: `add_face.py` — One-Shot user enrolment (Phase 2: encoding)

```
# --- PHASE 2: ENCODE FACES (after photo capture) ---
detector  = cv2.FaceDetectorYN.create(DETECTOR_MODEL, "", (320,320), 0.8, 0.3,
    5000)
recognizer = cv2.FaceRecognizerSF.create(RECOGNIZER_MODEL, "")

known_feats = []
known_names = []

for filename in os.listdir(KNOWN_FACES_DIR):
    if filename.endswith((".jpg", ".png", ".jpeg")):
        img = cv2.imread(os.path.join(KNOWN_FACES_DIR, filename))
        if img is None: continue

        height, width, _ = img.shape
        detector.setInputSize((width, height))
        _, faces = detector.detect(img)

        if faces is not None:
            aligned_face = recognizer.alignCrop(img, faces[0])
            feat         = recognizer.feature(aligned_face)
            known_feats.append(feat)
            # Extract clean name (strip '_N' suffix from filename)
            base_name = os.path.splitext(filename)[0]
            known_names.append(base_name.split('_')[0])

# Serialize to pickle
data = {"feats": known_feats, "names": known_names}
with open(ENCODINGS_FILE, "wb") as f:
    f.write(pickle.dumps(data))

print(f"SUCCESS! Database updated: {len(known_names)} embeddings stored.")
```

The operational workflow is thus reduced to two commands:

Listing 5: Code 5: Operational workflow

```
# To enrol a new user:
python3 add_face.py

# To run real-time recognition:
python3 main_deeplearning.py
```

6 Results and Analysis

The system was evaluated under a variety of real-world conditions including varying lighting, head orientations, and multiple simultaneous subjects.

6.1 Real-Time Performance

Table 4: Real-Time Performance Metrics

Metric	Target (KPI)	Achieved	Assessment
Frame Rate (FPS)	≥ 3 FPS	**3–4 FPS**	✓ Meets target
Inference Latency	< 300 ms	$\approx$ **250 ms**	✓ Meets target
True Positive Rate	$> 95\%$	$>$ **98%**	✓ Exceeds target
False Acceptance Rate (FAR)	$< 1\%$	$<$ **0.5%**	✓ Exceeds target

While 3–4 FPS is lower than desktop GPU systems, it is operationally sufficient for access-control scenarios such as a person walking towards a door — the system correctly identifies or rejects the individual before they reach the access point. The 250 ms latency is below the human perception threshold for reaction in access-control contexts.

6.2 Accuracy and Robustness

Table 5: Robustness Test Results

Test Condition	Detection Result	Notes
Frontal face, good lighting	Reliable	Confidence $> 90\%$
Side profile ($\pm 30^\circ$ yaw)	Reliable	YuNet landmark alignment compensates
Low-light environment	Consistent	Minor FPS reduction ($\sim$3 FPS)
Multiple registered faces	Simultaneous tracking	All faces tracked correctly
Unregistered face	Marked "Unknown"	Red bounding box, FAR $<$ 0.5%

6.3 Comparison with Previous Approach

Table 6: YuNet+SFace vs. Dlib/HOG (previous approach)

Criterion	Dlib/HOG	YuNet + SFace	Winner
Frame rate on Pi 4	$\sim$1 FPS	**3–4 FPS**	YuNet
Side-profile detection	Poor	**Up to** $\pm 30^\circ$	YuNet
Low-light robustness	Moderate	**Good**	YuNet
Installation complexity	Very high (dlib compile: 45 min)	**Simple (apt)**	YuNet
Registration speed	Re-train required	**Instant (One-Shot)**	YuNet
Cloud dependency	None	None	Tie

7 Discussion

1. Model Selection: The choice of YuNet over heavier detectors (RetinaFace, MTCNN) was critical for achieving usable FPS. YuNet's INT8 quantization reduces memory bandwidth requirements, which is the dominant bottleneck on the Pi's shared CPU-memory bus. SFace's 128-D embedding is intentionally small — larger embedding dimensions would increase cosine-similarity computation time proportionally.
2. The 2022mar vs. 2023mar Model Discrepancy: During development, the 2023mar YuNet model caused an "Unsupported layer" error in OpenCV 4.6.0 because it uses layer types introduced in ONNX opset 17, which is only supported from OpenCV 4.7.0 onwards. The 2022mar model (ONNX opset 11) is fully compatible with OpenCV 4.6.0 — the version installed via `apt` on Bookworm. This is an important practical finding for edge-deployment practitioners.
3. Frame Rate Ceiling: The 3–4 FPS ceiling is primarily set by the YuNet inference step ($\approx$180 ms) rather than SFace ($\approx$70 ms). Further optimisation could be achieved by: (a) reducing the input tensor from 320×320 to 240×240 (at the cost of detection sensitivity for small faces), or (b) integrating the Google Coral USB Accelerator, which would offload inference from the CPU entirely and is projected to raise FPS to 20+.
4. Security Threshold: The cosine similarity threshold of $\tau = 0.363$ was selected at the model's EER point. Increasing τ reduces the False Acceptance Rate (FAR) but increases the False Rejection Rate (FRR) — a standard security trade-off. For high-security applications, τ should be raised to 0.4–0.5, accepting that legitimate users may occasionally need a second attempt.
5. One-Shot vs. Traditional Training: The `add_face.py` enrolment script demonstrates the practical advantage of embedding-based recognition over classification-based approaches. A new user can be added in under 60 seconds without any Python re-execution or model retraining — a significant operational advantage for deployment in institutions with frequent staff turnover.

8 Relevance to National Development

This project directly supports the Government of Bangladesh's "Digital Bangladesh" vision and the "Smart Bangladesh 2041" roadmap by providing a low-cost, domestically producible biometric security solution. Specific application scenarios include:

- **Automated Attendance:** Schools, colleges, and government offices can deploy the system for contactless attendance logging (future integration with SQL database planned).
- **Secure Entry Control:** Offices and research laboratories can use the system for door access control without recurring cloud service fees.
- **Small Business Surveillance:** The total hardware cost of BDT 16,600 ($\approx$ USD 150) is accessible to small businesses, reducing reliance on expensive imported security infrastructure.
- **Data Sovereignty:** All biometric data remains on the local device — an important consideration under Bangladesh's emerging data-protection framework.

9 Future Scope

1. **Hardware Acceleration:** Integration with the Google Coral USB Accelerator (BDT $\approx$ 8,000) is projected to increase frame rate from 3–4 FPS to 20+ FPS by offloading inference from the CPU to the Edge TPU.

2. **Database Integration:** Linking the recognition output to an SQLite or MySQL database for time-stamped attendance logging with automatic report generation.

3. **Liveness Detection:** Implementing anti-spoofing measures to prevent false acceptance via 2D photographs or printed faces (planned using depth estimation or blink detection).

4. **Multi-Camera Support:** Extending the pipeline to support multiple cameras for wider field-of-view coverage in corridors and entrance lobbies.

5. **Web Dashboard:** A Flask-based web interface to allow remote monitoring of recognition logs and database management from a browser.

10 Conclusion

This project successfully designed and implemented a real-time, offline facial recognition system on a Raspberry Pi 4 using a quantized deep-learning pipeline. The following key conclusions are drawn:

1. **Performance:** The YuNet + SFace pipeline achieves 3–4 FPS with $\approx$250 ms inference latency on the Pi's native ARM CPU — without any external hardware accelerator — meeting and exceeding all defined Key Performance Indicators.

2. **Accuracy:** True positive rate exceeds 98% for registered users. The False Acceptance Rate is below 0.5% at the standard cosine threshold of 0.363, making the system suitable for access-control applications.

3. **Robustness:** The system reliably detects and recognizes faces under challenging conditions including side profiles (up to $\pm 30^{\circ}$ yaw), low-light environments, and simultaneous multi-face tracking.

4. **Ease of Enrolment:** The One-Shot `add_face.py` enrolment script registers a new user in under 60 seconds without any model retraining, making the system operationally practical.

5. **Privacy and Cost:** The entirely offline, edge-based architecture eliminates cloud dependency, protecting biometric data and eliminating recurring operational costs. The total hardware cost of BDT 16,600 represents a highly cost-effective solution for Bangladeshi institutions.

References

[1] F. Zhong, J. Deng, G. Yang, J. Chen, Y. Wei, and H. Zhang, "SFace: Sigmoid-Constrained Hypersphere Loss for Robust Face Recognition," *IEEE Transactions on Image Processing*, vol. 30, pp. 3687–3698, 2021.

[2] W. Wu, Y. Peng, and S. Yu, "YuNet: A Tiny Face Detector for Edge Devices," OpenCV Model Zoo, 2022. [Online]. Available: `https://github.com/opencv/opencv_zoo`

[3] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, vol. 25, pp. 120–123, 2000.

[4] Raspberry Pi Foundation, "Raspberry Pi 4 Model B Datasheet," 2019. [Online]. Available: https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf

[5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

[6] A. G. Howard *et al.*, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv preprint arXiv:1704.04861*, 2017.

[7] Raspberry Pi Foundation, "Picamera2 Library Documentation," 2023. [Online]. Available: https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf